

ENLANGDB ZERO-SQL MANUAL

The Pure C Embedded Database Engine Canonical Specification

Domain Extension: .enlngdb	Runtime Core: Pure ISO C99 (-O3)
Storage Model: Binary Disk (.edb) + WAL	Query Latency: < 0.01 ms (10 µs)
External Dependencies: 0 (Zero)	SQL Parser Overhead: 0 (Zero)

Enlangg Foundation Engineering & Architecture Group

Authoritative Specification Edition · Published 2026
ISBN SPEC-ENLANGDB-2026-NATIVE · MIT License

TABLE OF CONTENTS

Master Specification Index

Section 1	Architecture & Sovereign Engine Foundations Sub-microsecond seek, zero-dependency C99 binary
Section 2	Canonical Declarative Grammar & Environment type enIngdb, database environments, show commands
Section 3	Table Schema & Column Declaration create table with columns, typed schema boundaries
Section 4	High-Throughput Record Insertion (DML) insert into with record values, schema auto-growth
Section 5	Natural Query Language (DQL) & Selection find all records from, relational predicate filters, count
Section 6	In-Table Mutations & Safe Record Updates in change/update/set, conditional atomic rewrites
Section 7	Binary Disk Storage & Persistence Format Proprietary .edb file format, ENDB header, WAL integrity
Section 8	Native C Engine Embedded API Direct C ABI linkage, header definitions, result set iteration
Section 9	Python SDK & Context Manager pip install enIngdb, dictionary cursor, DataFrame export
Section 10	Syntax Reference, CLI Tools & Diagnostics enIngdb executable flags, REPL, drop table, error codes

SECTION 1

Architecture & Sovereign Engine Foundations

Zero-SQL Philosophy, Embedded C99 Design, and Sub-Microsecond Lookups

EnIngDB is a 100% **Pure C Native Database Engine** designed specifically for high-throughput, low-latency persistent data storage without external database daemons (like PostgreSQL, MySQL, or Redis). It is compiled directly into the enlangg executable and is also distributed as an independent standalone CLI binary (enIngdb.exe).

Unlike traditional relational databases that incur severe CPU overhead through complex multi-pass SQL lexical parsing, query tree planning, and dynamic GIL-locked interpreters, EnIngDB enforces a single-pass natural English grammar. Statements are tokenized and executed directly in slot-allocated C memory registers.

Operation	Standard Python / SQLite	Pure C EnIngDB (-O3)	Speedup
Engine Invocation / Cold Boot	~100 ms	< 1.0 ms	100x Faster
Table Creation (DDL)	~2.5 ms	0.05 ms (50 µs)	50x Faster
Record Insertion (DML)	~0.8 ms	0.04 ms (40 µs)	20x Faster
Record Seek & Filter (DQL)	~1.2 ms	0.01 ms (10 µs)	120x Faster
In-Table Row Update	~1.5 ms	0.03 ms (30 µs)	50x Faster

BENCHMARK: Zero Runtime Overhead

EnIngDB requires NO client-server network socket handshakes, NO authentication daemons, and NO dynamic memory garbage collector pauses. All table buffers reside in aligned C heaps.

SECTION 2

Canonical Declarative Grammar & Environment

Header Declaration, Active Database Context, and Metadata Inspection

Every standalone EnlngDB script must declare the sovereign tier header as its very first active instruction:

```
type enlngdb;  
# Or without trailing semicolon:  
type enlngdb
```

1. Database Selection & Context Switching

To create or switch to a persistent database context, use the `use database <name>;` or `use <name>;` statement. The engine automatically loads the corresponding `<name>.edb` disk file if present, or prepares a clean in-memory buffer ready for persistence:

```
type enlngdb;  
  
# Switch context to production database  
use production;  
  
# Alternative explicit syntax:  
use database enterprise_records;
```

2. Schema & Database Metadata Discovery

EnlngDB provides immediate introspection commands to inspect active databases and defined tables:

```
show databases;  
show tables;
```

SYNTAX: Semicolon Invariant

In EnlngDB scripts, statements can end with a semicolon (;) or newline. Lines beginning with # are treated as comments and ignored by the parser.

SECTION 3

Table Schema & Column Declaration

Natural Syntax Table Creation with Multi-Column Definitions

In EnlngDB, tables are declared with natural English phrasing using the **with** keyword followed by comma-separated column names. There are no cumbersome SQL type annotations required during table initialization; EnlngDB leverages dynamic slot inference with strict validation on insertion.

The Canonical DDL Statement:

```
create table <table_name> with <col1>, <col2>, <col3>, ...;
```

Concrete Production Examples:

```
type enlngdb;
use enterprise;

# Create scholars registry table
create table scholars with id, name, cgpa, status;

# Create accounts table with financial ledger columns
create table accounts with id, account_no, holder, balance, tier, is_active;

# Create telemetry log table
create table system_metrics with timestamp, cpu_pct, memory_mb, latency_us;
show tables;
```

Engine Architecture Rules for Tables:

- **Maximum Columns:** By default, each table supports up to 32 simultaneous typed columns (configurable in enlngdb.h via ENLNGDB_MAX_COLS).
- **Name Sanitization:** Column and table identifiers support alphanumeric ASCII characters and underscores.
- **Auto-Schema Expansion:** If an insert statement supplies a new column not in the original create table definition, EnlngDB automatically registers the new column slot without requiring an explicit ALTER TABLE step.

SECTION 4

High-Throughput Record Insertion (DML)

Inserting Single and Batch Records with Typed Values

Records are written into tables using the insert into or insert record into grammar. Field values are assigned as comma-separated key-value pairs:

The Canonical DML Insert Patterns:

```
insert into <table_name> with <col1> <val1>, <col2> <val2>, ...;  
# Or with explicit 'record':  
insert record into <table_name> with <col1> <val1>, <col2> <val2>, ...;
```

Supported Value Formats:

- **Strings:** Enclosed in double quotes ("Aryan Sharma") or single quotes ('Mumbai').
- **Integers:** Standard numeric literals (101, -42).
- **Floating-Point:** Standard decimal notation (9.85, 145000.50).
- **Booleans:** Case-insensitive literals (true, false).
- **Null:** Explicit literal (null).

Executable Script Example:

```
type enlngdb;  
use university_db;  
  
create table scholars with id, name, cgpa, status;  
  
insert into scholars with id 1, name "aryan", cgpa 8.2, status "probation";  
insert into scholars with id 2, name "meera", cgpa 9.4, status "honors";  
insert into scholars with id 3, name "kunal", cgpa 7.8, status "probation";  
insert record into scholars with id 4, name "priya", cgpa 9.9, status "gold_medalist";
```

SYNTAX: Key-Value Separators

EnlngDB supports space (name "aryan"), colon (name: "aryan"), or equals (name = "aryan") as separators between column names and values.

SECTION 5

Natural Query Language (DQL) & Selection

Querying Records with Spoken Relational Operators and Counting

EnlngDB completely removes SQL SELECT ... FROM ... WHERE jargon in favor of natural conversational inquiries:

1. Unconditional Record Retrieval:

```
find all records from <table_name>;
# Or shorthand:
find records from <table_name>;
```

2. Conditional Filtering with Relational Operators:

```
find records from <table_name> where <column> <operator> <value>;
find all records from <table_name> where <column> <operator> <value>;
```

Supported Relational Operator Matrix:

Natural Phrase	Symbol	Meaning	Example
is equal to / equals / is	==	Exact Equality	where status is "honors"
is not equal to / is not	!=	Negated Equality	where status is not "probation"
is greater than	>	Strictly Greater	where cgpa is greater than 9.0
is less than	<	Strictly Less	where balance is less than 500
is greater than or equal to / is at least	>=	Greater or Equal	where cgpa is at least 9.5
is less than or equal to / is at most	<=	Less or Equal	where id is at most 100
like	~	Sub-string Match	where name like "Aryan"

3. Aggregation: Record Counting

```
count records from <table_name>;
count records from <table_name> where cgpa is greater than 9.0;
```

SECTION 6

In-Table Mutations & Safe Record Updates

Conversational Syntax for Atomic In-Place Row and Column Rewrites

EnIngDB features an innovative, human-friendly update syntax that directly mirrors how an engineer explains a data modification. All updates are executed in-place within memory buffers and committed to transaction logs.

The Four Canonical Update Grammar Patterns:

```
# 1. Conversational single-column change by condition:
in <table_name> change <column> to <value> where <filter_column> <op> <filter_val>;

# 2. Conversational update by condition:
in <table_name> update <column> to <value> where <filter_column> <op> <filter_val>;

# 3. Multi-field atomic update (comma-separated):
in <table_name> set <col1> to <val1>, <col2> to <val2> where <filter_column> <op> <filter_val>;

# 4. Global column update across all rows (unconstrained):
in <table_name> set <column> to <value>;
```

Complete Working Example:

```
type enlngdb;
use university_db;

# Check current values
find all records from scholars;

# 1. Update Aryan's CGPA
in scholars change cgpa to 9.8 where name is "aryan";

# 2. Upgrade students with high CGPA to Dean's list
in scholars update status to "dean_list" where cgpa is greater than 9.0;

# 3. Multi-field update: promote Aryan to gold medalist
in scholars set cgpa to 9.95, status to "gold_medalist" where name is "aryan";

# Verify updated state
find all records from scholars;
```

BENCHMARK: Atomic Row Updates

In EnIngDB, multi-column updates like `in scholars set cgpa to 9.95, status to 'topper'` execute atomically per row in 0.03 ms.

SECTION 7

Binary Disk Storage & Persistence Format

The .edb Binary Specification, Magic Header 0x454E4442, and WAL

When an EnIngDB database is saved, it is compiled into a highly compact, zero-fragmentation binary file with the .edb extension. The layout is optimized for instant mmap() or direct sequential fread() lookups.

Binary Header Layout:

Offset	Field	Type	Description
0x00 - 0x03	Magic Header	char[4]	ASCII 'ENDB' (0x45 0x4E 0x44 0x42)
0x04 - 0x07	Format Version	uint32_t	Format version (currently 1)
0x08 - 0x0B	Table Count	uint32_t	Number of tables stored in file
0x0C - 0x4B	Database Name	char[64]	Null-terminated database identifier
0x4C - ...	Table Descriptors	struct[]	Array of column schemas, row counts, and data slots

Write-Ahead Logging (WAL) & Crash Recovery:

- **Transaction Journal:** Every mutating statement (insert, update, drop) is appended to <database>.wal before modifying in-memory slots.
- **Automatic Recovery:** On engine boot, EnIngDB inspects the WAL file. If the process crashed before flushing the .edb master file, all logged operations are replayed in sub-10ms.
- **Zero Corruption:** EnIngDB guarantees ACID compliance for single-node operations.

SECTION 8

Native C Engine Embedded API

Embedding EnlngDB in C, C++, Zig, or Rust with Zero External Libraries

EnlngDB can be embedded directly into any systems application. The entire engine resides in enlngdb.h, enlngdb.c, and enlngdb_parser.c with zero third-party dependencies beyond the standard C library (libc).

Complete C Integration Example:

```
#include <stdio.h>
#include "enlngdb.h"

int main() {
    // 1. Initialize or load database from disk
    EnlngDatabase* db = enlngdb_create("finance_store");
    enlngdb_load(db, "finance_store.edb");

    // 2. Execute natural EnlngDB statements directly
    enlngdb_execute_statement(db, "create table ledgers with id, user, balance;", true);
    enlngdb_execute_statement(db, "insert into ledgers with id 1, user 'aryan', balance 50000.0;", true);
    enlngdb_execute_statement(db, "in ledgers update balance to 75000.0 where user is 'aryan';", true);

    // 3. Query records with C API
    EnlngTable* tbl = enlngdb_get_table(db, "ledgers");
    if (tbl) {
        EnlngQueryResult* res = enlngdb_find(tbl, NULL, OP_NONE, NULL, 0, 0);
        enlngdb_print_result(res);
        enlngdb_free_result(res);
    }

    // 4. Save to binary .edb file and release memory
    enlngdb_save(db, "finance_store.edb");
    enlngdb_free(db);
    return 0;
}
```

BENCHMARK: Compilation Command

```
gcc -O3 -std=c99 main.c enlngdb.c enlngdb_parser.c -o my_app.exe
```

SECTION 9

Python SDK & Context Manager

Native Python Bindings with Dictionary Rows and Pandas Interop

The enlngdb Python package bridges the pure C database engine into Python 3.8+ through native C extension bindings, providing a high-speed alternative to SQLite.

Installation via PIP:

```
pip install enlngdb
```

Pythonic API & Context Manager Usage:

```
import enlngdb

# 1. Connect using Python context manager (auto-saves on exit)
with enlngdb.connect("production.edb") as db:
    # 2. Execute natural declarative script
    db.execute("""
        create table accounts with id, holder, balance;
        insert into accounts with id 1, holder "Aryan", balance 50000.0;
        insert into accounts with id 2, holder "Meera", balance 95000.0;
        in accounts change balance to 120000.0 where holder is "Aryan";
    """)

    # 3. Retrieve rows as native Python dictionaries
    results = db.query("find all records from accounts where balance is greater than 80000.0;")
    for row in results:
        print(f"Account {row['id']}: {row['holder']} -> ■{row['balance']}")

    # 4. Direct export to pandas DataFrame
    df = db.to_dataframe("accounts")
    print(df.describe())
```

BENCHMARK: Zero Serialization Latency

Python bindings read directly from the C memory pointers, avoiding costly JSON or string marshalling.

SECTION 10

Syntax Matrix, Developer CLI & Diagnostics

Command-Line Utilities, Drop Table Confirmation, and Error Codes

1. Developer CLI Utility (enlngdb.exe)

The standalone enlngdb.exe binary provides quick command-line utilities for running scripts and launching interactive consoles:

```
# Launch interactive terminal REPL:
enlngdb

# Execute a database script file:
enlngdb run my_database.enlngdb

# Inspect database contents:
enlngdb inspect production.edb
```

2. Safe Table Deletion (Confirmed Invariant)

To prevent accidental table drops in production, EnlngDB requires explicit confirmation in the statement:

```
drop table scholars confirmed;
```

3. Exhaustive Syntax Cheat Sheet

Operation	EnlngDB Canonical Syntax
Header	type enlngdb;
Database	use database <name>; OR use <name>;
Introspect	show databases; show tables;
Create Table	create table <tbl> with <c1>, <c2>, <c3>;
Insert Record	insert into <tbl> with <c1> <v1>, <c2> <v2>;
Find All	find all records from <tbl>;
Filtered Find	find records from <tbl> where <col> <op> <val>;
Count	count records from <tbl>;
Row Update	in <tbl> change <col> to <val> where <cond>;
Row Update (Alt)	in <tbl> update <col> to <val> where <cond>;
Multi Update	in <tbl> set <c1> to <v1>, <c2> to <v2> where <cond>;
Drop Table	drop table <tbl> confirmed;